

page replacement

to make room for **new pages** that are brought into memory when **page faults** occur, the operating system will need to sometimes **remove pages**

if a page to be replaced is *dirty* (and the page is not “file-backed”), written to **swap space** on disk

- can use one of the page table **status bits** to record whether dirty or not

OS needs to be able to do the *inverse mapping* to that done by a page table

- given a **physical page** that OS wants to replace the contents of, what **virtual pages** are mapped to it?
- OS needs to **invalidate page table entries** for removed pages and any copies of these in TLB (*how?*)

pages that are good candidates for replacement are identified **proactively**

... then, if a page replacement is needed, will have already identified some good choices

page replacement policies

typically operating system uses some *approximation* to LRU

e.g. “clock” algorithm (simplest variant):

- each page table entry includes a “reference” (or “use”) bit
- OS scans the physical pages at some steady rate, setting use bits of corresponding page table entries (*how found?*) to 0
- if when OS scan reaches a page the corresponding use bit is already 0, page is good candidate for replacement!

page table size

the problem: page tables too big if have one entry for each virtual page in the virtual address space

- many processors now support 57-bit virtual addresses (up from 48 bit) and 4 KiB = 2^{12} byte pages
- virtual address space then has $2^{57} / 2^{12} = 2^{45}$ virtual pages ... if one 8-byte PTE for each virtual page, get $2^{45} \times 8 \text{ bytes} = 256 \text{ TiB}$ for a single page table!

one solution: maintain **bounds** on the **ranges of virtual addresses in use**, only have page table entries for addresses within bounds

more flexible solution: **multiple levels of page table**

e.g. suppose **30-bit** virtual addresses, **4 KiB** pages

- divide the virtual pages into “**clusters**” of **512** (2^9) consecutive pages each

29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

- **bits 0-11** give which **byte within page**, **bits 12-20** which **page within page cluster**, **bits 21-29** which **cluster**
- **bits 21-29** used to index into a “root” page table with 512 (2^9) entries giving the physical page numbers of “leaf” page tables
- **bits 12-20** index into a “leaf” page table, each has 512 entries giving physical page numbers for virtual pages
- if each entry 8 bytes, each table fits in one 4 KiB page
- *at most* 512 leaf page tables ... only need to allocate a particular leaf table if process is using at least one page from that part of the virtual address space

with each additional level of tables, can increase virtual address size by 9 bits (*how many levels needed for 57-bit addresses?*)

neat feature: can support a **mix of page sizes** (e.g. **4 KiB**, **2 MiB** = **512 x 4 KiB**); e.g., in two-level scheme, a “root” page table entry could (optionally) give a starting PPN for whole cluster

(System) Virtual machines

(reference: text Section 5.6)

idea to use a **VMM** (“**virtual machine monitor**”, or “**hypervisor**”) to support sharing of the resources of a computer among a number of **VMs** (“**virtual machines**”)

each VM includes its own operating system, and presents to its users the illusion of having an entire computer to themselves

... *motivation?* can run **multiple unrelated applications/services** on the **same computer**, even if they need different OSes, with **improved fault isolation and security**

use cases include:

- **server consolidation** (replacing multiple servers by a single server)
- **cloud computing, IaaS** (“**Infrastructure as a Service**”)
- ... and other **scenarios where one wants strong fault isolation / security**

implementing the virtual machine idea

only the VMM must be allowed direct access to shared system resources, such as I/O devices

in system without VMs:

- application programs run in “user mode”
- operating system runs in “kernel” (“supervisor”) mode

in system with VMs, one approach is to run *both* the OS of each VM (a “guest OS”) and the applications in user mode, and the VMM in kernel mode

idea is that when a guest OS attempts a privileged operation, an exception will occur, causing the VMM to be invoked ... but some pitfalls:

- some architectures have cases where *don't* get exception
- can be substantial performance cost

how to address pitfalls in this approach?

- in “full paravirtualization”, modifications are made to guest OSes to *directly request* services from VMM
- hardware support for virtualization (e.g., new mode, “nested paging”, new instructions, ...)

typically: use hardware support, with paravirtualized drivers (rest of guest unmodified) (e.g., KVM-QEMU) or SR-IOV for I/O